Ex. No. 6	PL/SQL - FUNCTIONS AND PROCEDURES
Date of Exercise	09/09/2026

Aim

To create PL/SQL blocks, procedures and functions for various operations.

Description

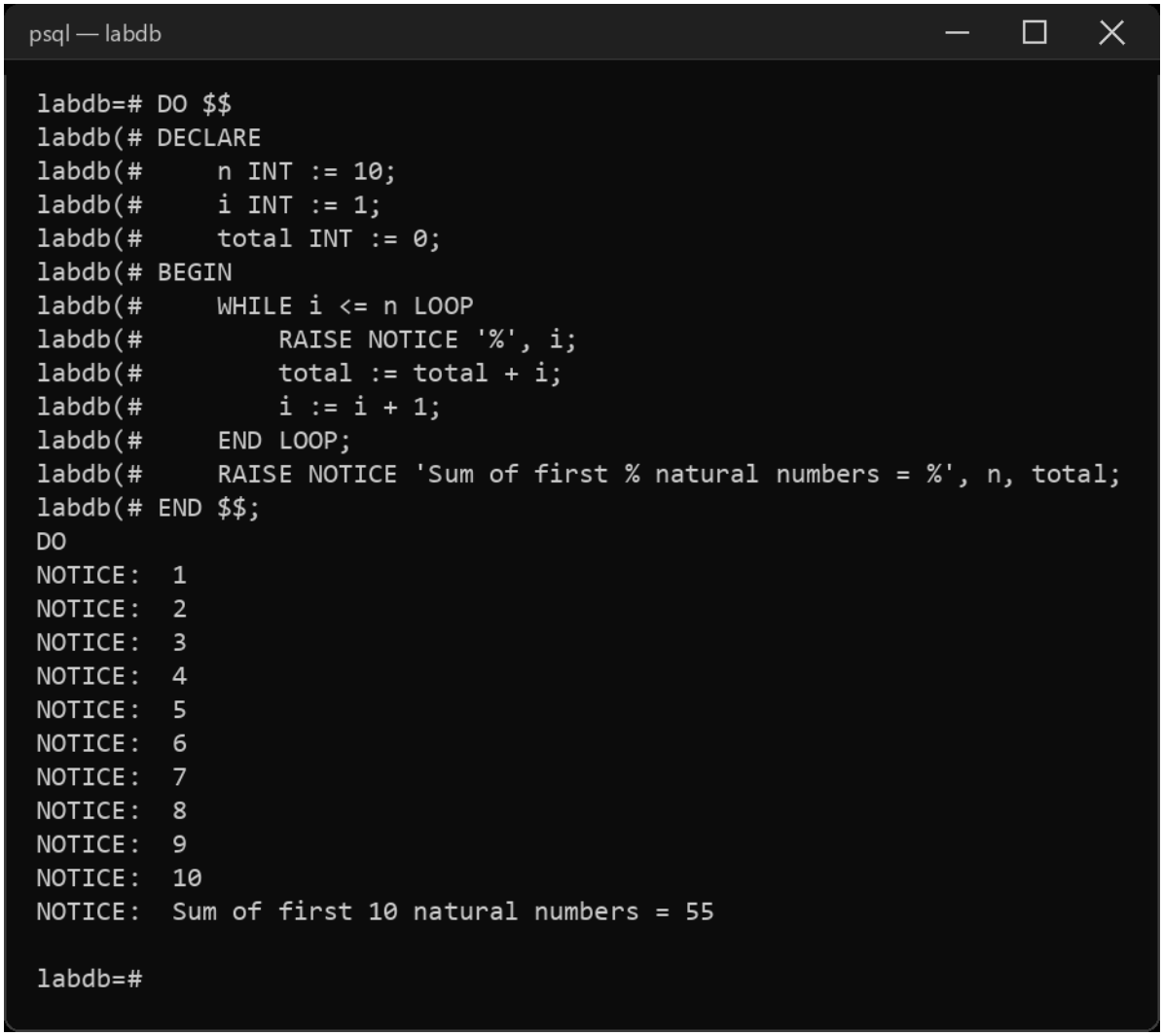
A PL/SQL block combines SQL with procedural constructs such as variables, loops and conditional statements. The optional DECLARE section defines variables and constants; the mandatory BEGIN...END section holds the executable logic, including DML statements, loops and IF/CASE conditionals; and the optional EXCEPTION section handles run-time errors raised during execution. A PROCEDURE is a named, storable PL/SQL block invoked with CALL, while a FUNCTION is a named block that must RETURN a value and can be used directly inside a SQL expression.

1. Write a PL/SQL block to print first n natural numbers and their sum.

Sample Code:

```
DO $$
DECLARE
    n INT := 10;
    i INT := 1;
    total INT := 0;
BEGIN
    WHILE i <= n LOOP
        RAISE NOTICE '%', i;
        total := total + i;
        i := i + 1;
    END LOOP;
    RAISE NOTICE 'Sum of first % natural numbers = %', n, total;
END $$;
```

Sample Output:

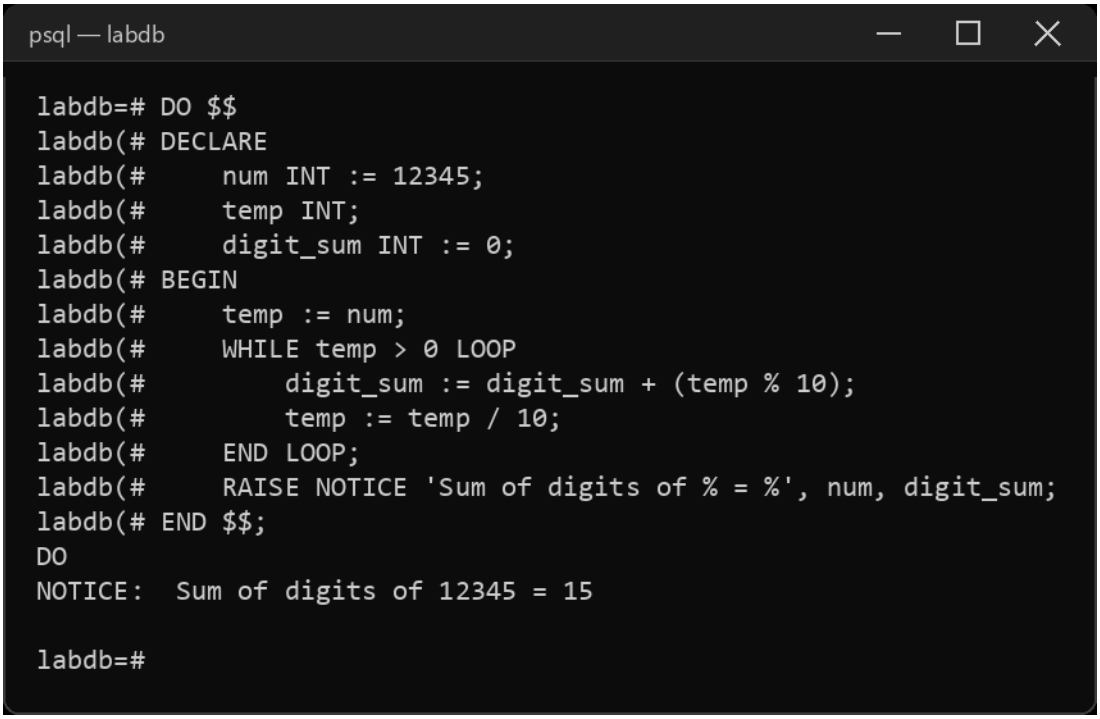


2. Write a PL/SQL block to find out sum of the digits of a given number.

Sample Code:

```
DO $$
DECLARE
    num INT := 12345;
    temp INT;
    digit_sum INT := 0;
BEGIN
    temp := num;
    WHILE temp > 0 LOOP
        digit_sum := digit_sum + (temp % 10);
        temp := temp / 10;
    END LOOP;
    RAISE NOTICE 'Sum of digits of % = %', num, digit_sum;
END $$;
```

Sample Output:

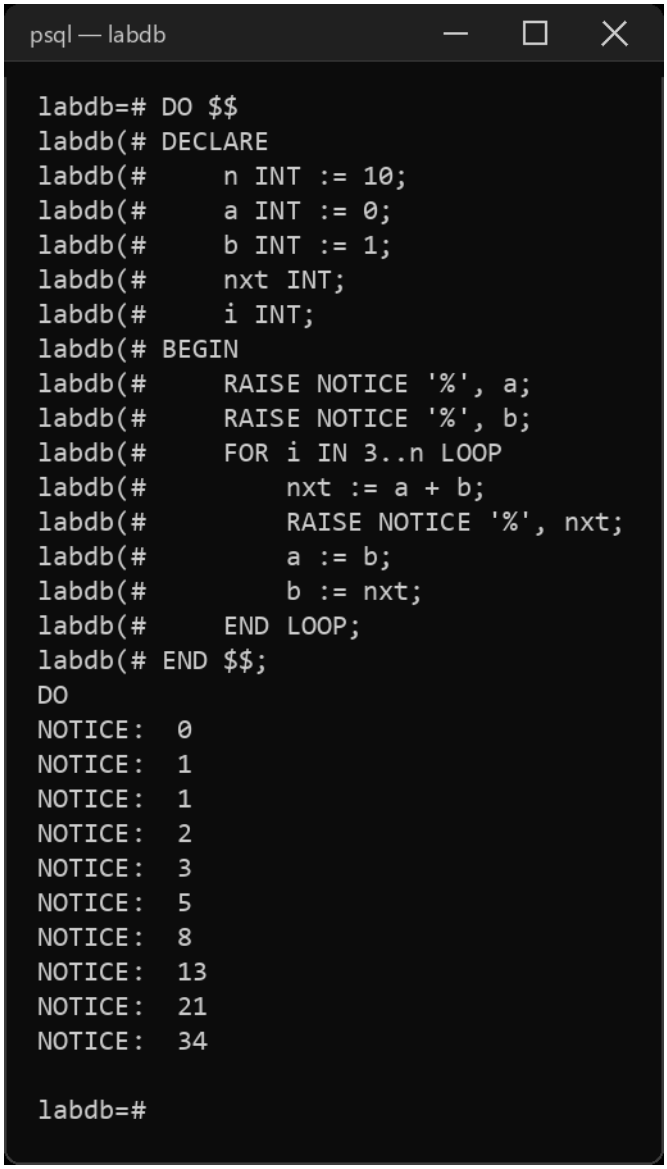


3. Write a PL/SQL block to generate a Fibonacci series.

Sample Code:

```
DO $$
DECLARE
    n INT := 10;
    a INT := 0;
    b INT := 1;
    nxt INT;
    i INT;
BEGIN
    RAISE NOTICE '%', a;
    RAISE NOTICE '%', b;
    FOR i IN 3..n LOOP
        nxt := a + b;
        RAISE NOTICE '%', nxt;
        a := b;
        b := nxt;
    END LOOP;
END $$;
```

Sample Output:

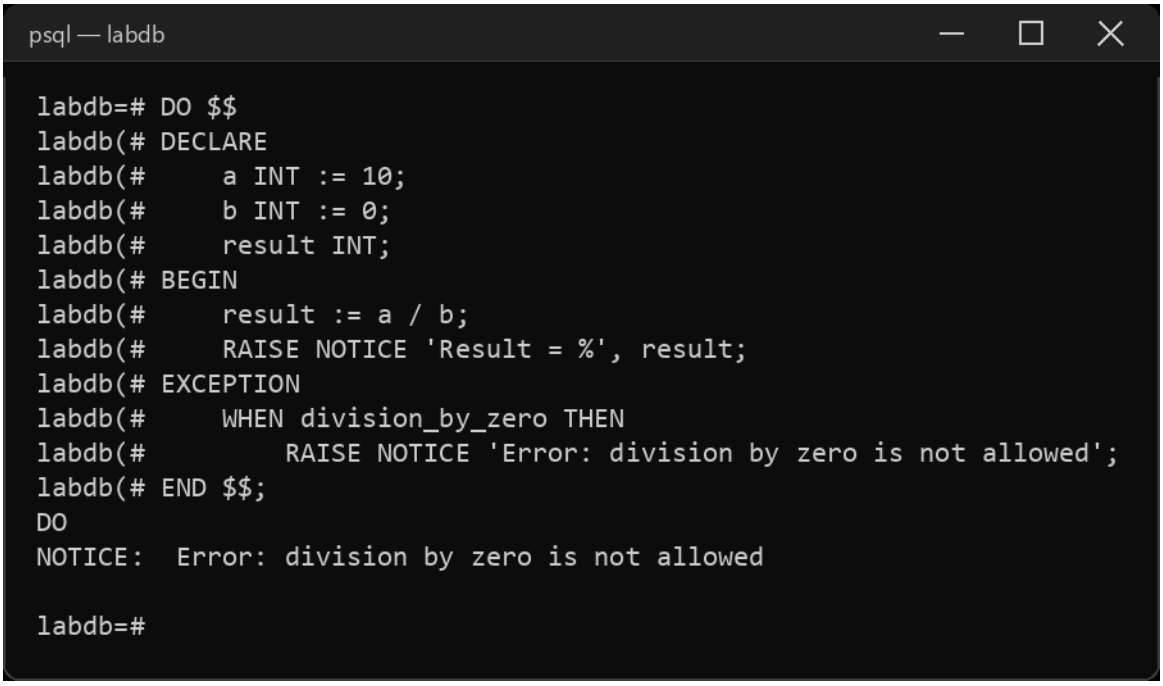


4. Write a PL/SQL script that traps ZERO_DIVIDE exceptions.

Sample Code:

```
DO $$
DECLARE
    a INT := 10;
    b INT := 0;
    result INT;
BEGIN
    result := a / b;
    RAISE NOTICE 'Result = %', result;
EXCEPTION
    WHEN division_by_zero THEN
        RAISE NOTICE 'Error: division by zero is not allowed';
END $$;
```

Sample Output:

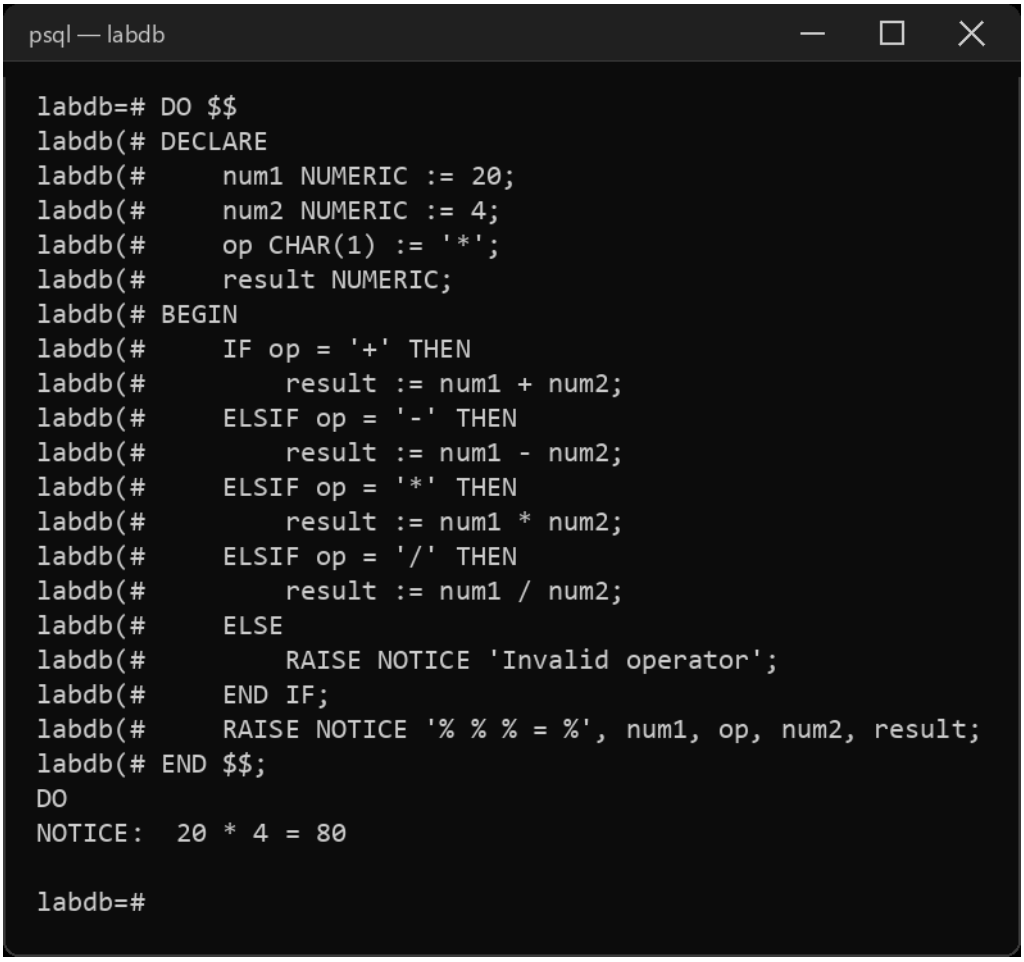


5. Write a PL/SQL block to illustrate functioning of IF_THEN_ELSIF by creating the equivalent of a four-function calculator.

Sample Code:

```
DO $$
DECLARE
    num1 NUMERIC := 20;
    num2 NUMERIC := 4;
    op CHAR(1) := '*';
    result NUMERIC;
BEGIN
    IF op = '+' THEN
        result := num1 + num2;
    ELSIF op = '-' THEN
        result := num1 - num2;
    ELSIF op = '*' THEN
        result := num1 * num2;
    ELSIF op = '/' THEN
        result := num1 / num2;
    ELSE
        RAISE NOTICE 'Invalid operator';
    END IF;
    RAISE NOTICE '% % % = %', num1, op, num2, result;
END $$;
```

Sample Output:



6. Write a PL/SQL Procedure to find the area of the circle.

Sample Code:

```
CREATE OR REPLACE PROCEDURE URK24CS1021_circle_area(radius NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
    pi_val CONSTANT NUMERIC := 3.14159265358979;
    area NUMERIC;
BEGIN
    area := pi_val * radius * radius;
    RAISE NOTICE 'Area of circle with radius % = %', radius, round(area, 2);
END;
$$;
CALL URK24CS1021_circle_area(7);
```

Sample Output:

```
psql — labdb

labdb=# CREATE OR REPLACE PROCEDURE URK24CS1021_circle_area(radius NUMERIC)
labdb(# LANGUAGE plpgsql AS $$
labdb(# DECLARE
labdb(#     pi_val CONSTANT NUMERIC := 3.14159265358979;
labdb(#     area NUMERIC;
labdb(# BEGIN
labdb(#     area := pi_val * radius * radius;
labdb(#     RAISE NOTICE 'Area of circle with radius % = %', radius, round(area, 2);
labdb(# END;
labdb(# $$;
CREATE PROCEDURE

labdb=# CALL URK24CS1021_circle_area(7);
CALL
NOTICE:  Area of circle with radius 7 = 153.94

labdb=#
```

7. Write a PL/SQL Function to find the area of the rectangle.

Sample Code:

```
CREATE OR REPLACE FUNCTION URK24CS1021_rectangle_area(length NUMERIC, width
NUMERIC)
RETURNS NUMERIC LANGUAGE plpgsql AS $$
DECLARE
    area NUMERIC;
BEGIN
    area := length * width;
    RETURN area;
END;
$$;
SELECT URK24CS1021_rectangle_area(5, 10) AS rectangle_area;
```

Sample Output:

```
psql — labdb

labdb=# CREATE OR REPLACE FUNCTION URK24CS1021_rectangle_area(length NUMERIC, width NUMERIC)
labdb(# RETURNS NUMERIC LANGUAGE plpgsql AS $$
labdb(# DECLARE
labdb(#     area NUMERIC;
labdb(# BEGIN
labdb(#     area := length * width;
labdb(#     RETURN area;
labdb(# END;
labdb(# $$;
CREATE FUNCTION

labdb=# SELECT URK24CS1021_rectangle_area(5, 10) AS rectangle_area;
 rectangle_area
----------------
             50
(1 row)

labdb=#
```

RESULT

The SQL queries were executed and the desired output was obtained.

EX6: PL/SQL - FUNCTIONS AND PROCEDURES